XML

CHEAT SHEETS

A Complete Reference for Structured Documents & Data

Document Structure · Elements & Attributes · CDATA & Entities · Namespaces · DTD · XML Schema · XPath · XSLT · DOM & SAX · SOAP · RSS · SVG · XXE Security · XSLT Grouping · XPath 3.1 & XQuery · plist · OOXML · JUnit

16 REFERENCE SHEETS

CONTENTS

FUNDAMENTALS

01 Basics & Document Structure
02 Elements & Attributes
03 Text, Data & Special Characters
04 Namespaces
05 DTD — Document Type Definition

VALIDATION & QUERYING

06 XML Schema (XSD)
07 XPath
08 XSLT

PROCESSING & PRACTICE

09 Parsing XML
10 Practical XML

SECURITY & BEYOND

11 XML Security
12 XSLT — Variables, Keys & Grouping
13 XPath 2.0 / 3.1 & XQuery Basics
14 XML Formats in the Wild

QUICK REFERENCE

15 Quick Reference — Syntax & Validation
16 Quick Reference — XPath, XSLT & Parsing

Covers XML 1.0 · Namespaces · DTD · XSD · XPath 1.0 · XSLT 1.0 (2026)

Begin → Sheet 01

Table of Contents

16 reference sheets · syntax · examples · notes

Sheet 1
Basics & Document Structure
prolog · declaration · root element · well-formedness · parsers · file types

Sheet 2
Elements & Attributes
element syntax · nesting · empty elements · attributes · element vs attribute

Sheet 3
Text, Data & Special Characters
CDATA · entities · character refs · whitespace · encoding · Unicode

Sheet 4
Namespaces
xmlns · prefixes · default namespace · scope · XHTML · SOAP · SVG

Sheet 5
DTD — Document Type Definition
!ELEMENT · !ATTLIST · !ENTITY · content models · internal · external

Sheet 6
XML Schema (XSD)
schema structure · simple types · complex types · restrictions · xs:sequence

Sheet 7
XPath
axes · node selection · predicates · functions · absolute · relative paths

Sheet 8
XSLT
templates · apply-templates · for-each · if · choose · output methods

Sheet 9
Parsing XML
DOM · SAX · StAX · Python · JS · tree navigation · error handling

Sheet 10
Practical XML
SOAP · REST · RSS · SVG · config files · Maven · Android · XML to JSON

Sheet 11
XML Security
XXE · billion laughs · external entities · disallow-doctype-decl · defusedxml · XPath injection

Sheet 12
XSLT — Variables, Keys & Grouping
xsl:variable · xsl:param · xsl:key · key() · mode · xsl:for-each-group · Muenchian grouping

Sheet 13
XPath 2.0 / 3.1 & XQuery Basics
sequences · if / then / else · for · let · ! · || · => · maps & arrays · FLWOR

Sheet 14
XML Formats in the Wild
plist · .docx / .xlsx (OOXML) · sitemap.xml · Atom · .csproj · JUnit XML · GPX / KML

Sheet 15
Quick Reference — Syntax & Validation
well-formedness · elements & attributes · entities & CDATA · namespaces · DTD · XSD types & structure

Sheet 16
Quick Reference — XPath, XSLT & Parsing
XPath axes & functions · XSLT templates & control · parsers by language · DOM vs SAX vs StAX · xmllint · security

Sheet 01

Basics & Document Structure

prolog · declaration · root element · well-formedness · parsers · file types

WHAT IS XML?

XML (eXtensible Markup Language) is a text-based format for storing and transporting structured data. Unlike HTML, XML has no predefined tags — you define your own tag names to describe your data. Every XML document must be **well-formed**: it must have exactly one root element, all tags must be properly nested and closed, and attribute values must be quoted. XML is case-sensitive: <Book> and <book> are different elements.

KEY FACTS

- Defined by the W3C in 1998; current spec is XML 1.0 (5th ed.)
- File extensions: `.xml`, `.xsl`, `.xsd`, `.svg`, `.rss`
- Default encoding is UTF-8; declare others in the prolog
- Whitespace outside elements is preserved by default
- Comments use `<!-- text -->` syntax; cannot nest
- Processing instructions use `<?target data?>`
- Common parsers: browser DOM, Python `xml.etree`, Java JAXP

ANATOMY OF AN XML DOCUMENT

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XML prolog ↑ — always line 1 if present -->

<!-- Optional DOCTYPE declaration -->
<!DOCTYPE catalog SYSTEM "catalog.dtd">

<catalog>                          <!-- root element (exactly one) -->

  <book id="bk101" lang="en">      <!-- element with attributes -->
    <title>XML Developer's Guide</title>
    <author>Gambardella, Matthew</author>
    <price currency="USD">44.95</price>
  </book>

  <book id="bk102" lang="en">
    <title>Midnight Rain</title>
    <author>Ralls, Kim</author>
    <price currency="USD">5.95</price>
    <note />                       <!-- empty element (self-closing) -->
  </book>

</catalog>
```

NOTES

- → `<?xml?>` — prolog; optional but strongly recommended; must be first line with no whitespace before it
- → `version="1.0"` — only version in wide use; XML 1.1 exists but is rarely needed
- → `encoding="UTF-8"` — omitting this defaults to UTF-8; declare explicitly for clarity
- → `<catalog>` — every document must have exactly one root element wrapping all content
- → Attributes must always be quoted — single or double quotes both valid
- → `<note />` — self-closing form for empty elements; equivalent to `<note></note>`
- → Tags are case-sensitive: opening and closing tags must match exactly

WELL-FORMEDNESS RULES

RULE	EXAMPLE — WRONG → RIGHT
One root element	<a/> → wrap in <root>
Tags must close	text → text
Proper nesting	<a> → <a>
Quoted attributes	id=42 → id="42"
Case-sensitive tags	<Book>…</book> → match case
Escape special chars	< in text → <
Unique attr names	id="a" id="b" → only one id

COMMON XML FORMATS

FORMAT	USE CASE
SVG	Scalable vector graphics
RSS / Atom	News & podcast feeds
SOAP	Web service messages
XHTML	HTML rewritten as XML
pom.xml	Maven project config
AndroidManifest	Android app config
Office Open XML	.docx, .xlsx internals
KML	Geographic data (Google Maps)

COMMON MISTAKES

- Whitespace or a BOM before the `<?xml?>` prolog — parsers will reject the document
- Using bare `<` or `&` in text content — must escape as `<` / `&`
- Mismatched tag case: `<Title>…</title>` is not valid XML

TIPS

- Always declare `encoding="UTF-8"` in the prolog, even though it's the default — it prevents encoding surprises when files are moved between systems
- Validate with an online XML linter or your editor before parsing programmatically
- Use attributes for metadata (IDs, types) and child elements for content that may grow

SUMMARY

- `<?xml version="1.0" encoding="UTF-8"?>` — prolog goes on line 1
- One root element wraps all content — no siblings at the top level
- Well-formed = closed tags, proper nesting, quoted attributes, escaped specials
- XML is case-sensitive; opening and closing tags must match exactly

Elements & Attributes

ELEMENTS AND ATTRIBUTES

An **element** is the primary building block of XML, consisting of a start tag, content, and an end tag: `<price>44.95</price>`. Elements can contain text, other elements, or both (mixed content). An **attribute** provides additional metadata directly on the start tag in *name="value"* form. The key design question is always: should this be an element or an attribute? Use elements for data that might expand, repeat, or need child structure; use attributes for concise, single-value metadata like IDs and types.

ELEMENT NAMING RULES

- Names must start with a letter or underscore (`_`)
- Can contain letters, digits, hyphens, underscores, periods
- Cannot start with a digit or hyphen
- Cannot start with `xml` (any case) — reserved
- No spaces in names
- Case-sensitive: `<Book>` ≠ `<book>`
- Convention: use camelCase or kebab-case consistently

ELEMENT & ATTRIBUTE SYNTAX

```
<!-- Basic element: start tag · content · end tag -->
<title>XML Developer's Guide</title>

<!-- Empty element — two equivalent forms -->
<br />
<br></br>

<!-- Element with attributes -->
<book id="bk101" lang="en" available="true">
  <title>XML Developer's Guide</title>
  <price currency="USD">44.95</price>
</book>

<!-- Mixed content (text + child elements) -->
<description>
  A <em>comprehensive</em> guide to XML.
</description>

<!-- Multiple same-name children (repetition is fine) -->
<catalog>
  <tag>fiction</tag>
  <tag>reference</tag>
  <tag>xml</tag>
</catalog>
```

NOTES

- → `<br />` — self-closing syntax; the space before `/` is optional but improves compatibility with older parsers
- → `id="bk101"` — attribute order on a tag is not significant; parsers may reorder them
- → Attribute values must always be quoted — single or double quotes are both valid, but pick one style and be consistent
- → An element cannot have two attributes with the same name
- → Mixed content (text and child elements together) is valid but makes XPath and schema validation more complex — avoid it when possible
- → Repeated child elements like `<tag>` are idiomatic XML for lists

ELEMENT VS. ATTRIBUTE — WHEN TO USE

USE ELEMENT	USE ATTRIBUTE
Content that can repeat	Single, non-repeating value
Data with child structure	Simple scalar (ID, code, flag)
Human-readable content	Machine-readable metadata
Data you query with XPath	Classifiers and identifiers
Values that may grow/change	Values fixed by schema/type
`<price>44.95</price>`	currency="USD"

CONTENT MODELS

MODEL	DESCRIPTION
Element content	Contains only child elements, no direct text
Text content	Contains only text (character data)
Mixed content	Contains both text and child elements
Empty	No content; self-closing or open+close
Any	Any combination (used in flexible schemas)

COMMON MISTAKES

- Duplicate attribute names on the same element — `id="a" id="b"` is invalid; only one per name allowed
- Starting element names with a digit: `<1item>` is invalid; use `<item1>`
- Unquoted attribute values: `id=42` fails — must be `id="42"`

TIPS

- Always add an `id` or `key` attribute on repeating elements — makes XPath selection and merging far easier later
- Avoid mixed content models unless you're authoring document-style XML (like DocBook or DITA)
- Stick to one attribute quoting style across the whole document — double quotes are the convention

SUMMARY

- Elements hold data; attributes hold metadata on elements
- Empty elements can self-close: `<tag />`
- Attribute values must always be quoted; no duplicate names on one element
- Use elements for repeatable/expandable data, attributes for scalar identifiers

Text, Data & Special Characters

TEXT CONTENT & ESCAPING

Text inside XML elements is called **character data (CDATA)**. Five characters have special meaning to the XML parser and must be escaped when they appear in text content or attribute values: <, >, &, ', and ". For longer blocks of text containing many special characters — such as embedded code or HTML — wrap the block in a **CDATA section** (`<![CDATA[ … ]]>`), which tells the parser to treat the entire block as raw text. Character references let you insert any Unicode code point directly using either decimal (`A`) or hex (`A`) notation.

PREDEFINED ENTITIES

- `<` → `<` (less-than)
- `>` → `>` (greater-than)
- `&` → `&` (ampersand)
- `'` → `'` (apostrophe)
- `"` → `"` (double quote)
- `A` → `A` (decimal char ref)
- `A` → `A` (hex char ref)

ESCAPING & CDATA EXAMPLES

```
<!-- ── Escaped special characters ──────────────────────── -->
<formula>a &lt; b &amp;&amp; b &gt; c</formula>
<!-- renders as:  a < b && b > c -->

<title lang=&apos;en&apos;>O&apos;Brien</title>

<!-- ── CDATA section — no escaping needed inside ─────────── -->
<script>
  <![CDATA[
    if (a < b && c > d) {
      console.log("raw HTML & JS here");
    }
  ]]>
</script>

<!-- ── Character references ──────────────────────────────── -->
<symbol>&#169;</symbol>      <!-- © -->
<symbol>&#x00A9;</symbol>    <!-- © (hex) -->
<symbol>&#8364;</symbol>     <!-- € -->

<!-- ── Whitespace in text content ────────────────────────── -->
<desc xml:space="preserve">  indented   text  </desc>
```

NOTES

- → `<![CDATA[]]>` — the sequence `]]>` cannot appear inside a CDATA block; split into two sections if needed
- → `<` is mandatory in text; `>` is only strictly required after `]]` but escaping it everywhere is best practice
- → `'` and `"` — only strictly required inside attribute values using the matching quote character
- → Decimal char refs (`©`) and hex refs (`©`) are equivalent; hex is more common for Unicode
- → `xml:space="preserve"` — instructs processors to preserve all whitespace in this element; default behavior is application-defined
- → Encoding declared in the prolog must match the actual file bytes — UTF-8 is safe for all Unicode

WHITESPACE HANDLING

CONTEXT	BEHAVIOR
`Text content`	Preserved exactly as written by default
`Between elements`	Technically preserved; often ignored by apps
`xml:space="preserve"`	Signals processor to keep all whitespace
`xml:space="default"`	Application may normalize whitespace
`Attribute values`	Leading/trailing spaces stripped by parser
`CDATA sections`	All whitespace preserved verbatim

COMMON ENCODINGS

ENCODING	NOTES
`UTF-8`	Default; handles all Unicode; recommended
`UTF-16`	Fixed 2-byte (BOM required); used in some Windows APIs
`ISO-8859-1`	Latin-1; legacy Western European documents
`US-ASCII`	7-bit only; subset of UTF-8
`windows-1252`	Windows legacy; superset of Latin-1

COMMON MISTAKES

- Using bare `&` in text — e.g. `AT&T` must be `AT&T` or parser throws
- Embedding `]]>` inside a CDATA block — split the section at that point
- Declaring `encoding="UTF-8"` but saving the file as UTF-16 — the mismatch causes parse failures

TIPS

- Use CDATA sections for any content containing HTML, code snippets, or SQL — it eliminates the need to escape every special character
- Prefer UTF-8 for all new XML documents — it is universally supported and the XML default
- Add `xml:space="preserve"` to elements where indentation or line breaks carry meaning

SUMMARY

- 5 predefined entities: `< > & ' "`
- `<![CDATA[…]]>` — raw text block; no escaping needed inside
- Char refs: `©` (decimal) or `©` (hex) for any Unicode point
- UTF-8 is the default and recommended encoding for all XML

Namespaces

WHAT ARE XML NAMESPACES?

Namespaces prevent element name collisions when combining XML vocabularies from different sources — for example, merging an HTML `<table>` with a furniture `<table>`. A namespace is a URI that uniquely identifies a vocabulary. It is declared with the `xmlns` attribute: `xmlns="URI"` for a **default namespace** (applies to all unprefixed elements in scope) or `xmlns:prefix="URI"` for a **prefixed namespace**. The URI is just an identifier — it does not need to resolve to a real web page.

NAMESPACE RULES

- A namespace URI is an identifier, not a URL — it need not be accessible
- Namespace declarations are inherited by all descendant elements
- A prefix can be overridden on a child element
- Default namespace does *not* apply to attributes — attributes without a prefix have no namespace
- The prefix `xml` is predefined and always maps to `http://www.w3.org/XML/1998/namespace`
- The prefix `xmlns` itself is reserved and cannot be used as a regular prefix

NOTES

→ `xmlns="URI"` — default namespace; all unprefixed child elements inherit this namespace automatically

→ `xmlns:h="URI"` — binds prefix `h` to the URI; elements must be written as `<h:element>`

→ Prefixes are arbitrary shorthand — `h`, `html`, and `xhtml` all work if bound to the same URI

→ Two elements with the same local name but different namespace URIs are completely different elements

→ `xmlns:soap="..."` in the SOAP example — declaring on the root is idiomatic; avoids repeating on every element

→ To un-declare a default namespace on a child: `xmlns=""`

NAMESPACE SYNTAX

```
<!-- ── Default namespace ──────────────────────────────── -->
<html xmlns="http://www.w3.org/1999/xhtml">
  <head><title>Page</title></head>
</html>
<!-- all unprefixed elements belong to XHTML namespace -->

<!-- ── Prefixed namespaces ────────────────────────────── -->
<root
  xmlns:h="http://www.w3.org/1999/xhtml"
  xmlns:f="http://example.com/furniture">

  <h:table>             <!-- HTML table -->
    <h:tr><h:td>Cell</h:td></h:tr>
  </h:table>

  <f:table>             <!-- furniture table -->
    <f:width>80cm</f:width>
    <f:length>120cm</f:length>
  </f:table>

</root>

<!-- ── SOAP envelope (real-world example) ─────────────── -->
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <GetPrice xmlns="http://example.com/prices">
      <Item>Widget</Item>
    </GetPrice>
  </soap:Body>
</soap:Envelope>
```

COMMON NAMESPACE URIS

PREFIX	URI	USED IN
`xmlns`	http://www.w3.org/1999/xhtml	XHTML
`svg`	http://www.w3.org/2000/svg	SVG graphics
`xs / xsd`	http://www.w3.org/2001/XMLSchema	XML Schema
`xsi`	http://www.w3.org/2001/XMLSchema-instance	Schema instances
`soap`	http://schemas.xmlsoap.org/soap/envelope/	SOAP 1.1
`xsl`	http://www.w3.org/1999/XSL/Transform	XSLT stylesheets
`dc`	http://purl.org/dc/elements/1.1/	Dublin Core metadata
`rss`	http://purl.org/rss/1.0/	RSS 1.0 feeds

COMMON MISTAKES

- Assuming the default namespace applies to attributes — it doesn't; attributes without a prefix have no namespace
- Using the same prefix for different URIs in the same document — prefix-URI bindings must be consistent within a scope
- Treating the namespace URI as a URL that must resolve — it's just a unique string identifier

TIPS

- Declare all namespaces on the root element — it keeps them visible in one place and avoids repeated declarations
- Use well-known prefixes (`xs`, `xsi`, `soap`) to match community convention and aid readability
- When writing XPath against namespaced XML, you must register prefix-URI bindings in your XPath engine too

SUMMARY

- `xmlns="URI"` — default namespace for all unprefixed elements in scope
- `xmlns:prefix="URI"` — binds a prefix; elements use `prefix:name`
- Namespace URIs are identifiers only — they don't need to resolve
- Default namespace does not apply to attributes

Sheet 05

DTD — Document Type Definition

!ELEMENT · !ATTLIST · !ENTITY · content models · internal · external

WHAT IS A DTD?

A **Document Type Definition (DTD)** defines the legal structure of an XML document: which elements are allowed, in what order, how many times, and what attributes they carry. A DTD can be **internal** (embedded in the document's `<!DOCTYPE>` declaration) or **external** (a separate `.dtd` file referenced by URI). DTDs use their own non-XML syntax with declarations starting with `<!`. While DTDs were the original XML validation mechanism, XML Schema (XSD) has largely replaced them in modern workflows because XSD supports data types and namespaces.

DTD LIMITATIONS

- No data type support — all values are treated as text strings
- No namespace awareness
- Cannot constrain element content to a specific number pattern (e.g. exactly 3)
- Non-XML syntax — harder to process with standard XML tools
- External DTDs must be fetched at validation time (network cost)
- XSD or Relax NG preferred for new projects

NOTES

- → `(book+)` — content model; `+` means one or more; see occurrence table below
- → `#PCDATA` — Parsed Character Data; element contains text (may include entity references)
- → `!ATTLIST` — declares attributes for an element; can list multiple attributes in one declaration
- → `ID` — attribute type meaning the value must be unique across the document
- → `#REQUIRED` — attribute must be present; `#IMPLIED` = optional; `#FIXED "val"` = fixed value
- → `!ENTITY pub "..."` — defines a reusable text entity; referenced as `&pub;` anywhere in the document
- → `SYSTEM` vs `PUBLIC` — SYSTEM is a private URI; PUBLIC identifies a well-known DTD with a fallback URI

DTD SYNTAX

```
<!-- ── Internal DTD ──────────────────────────────────── -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE catalog [
  <!ELEMENT catalog  (book+)>
  <!ELEMENT book     (title, author, price, note?)>
  <!ELEMENT title    (#PCDATA)>
  <!ELEMENT author   (#PCDATA)>
  <!ELEMENT price    (#PCDATA)>
  <!ELEMENT note     (#PCDATA)>

  <!ATTLIST book
    id        ID       #REQUIRED
    lang      CDATA    "en"
    available (yes|no) "yes"
  >

  <!ENTITY pub "Publisher Name Inc.">
]>

<catalog>
  <book id="bk101" lang="en">
    <title>XML Guide</title>
    <author>&pub;</author>
    <price>44.95</price>
  </book>
</catalog>

<!-- ── External DTD ──────────────────────────────────── -->
<!DOCTYPE catalog SYSTEM "catalog.dtd">
<!DOCTYPE html PUBLIC
  "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

CONTENT MODEL SYMBOLS

SYMBOL	MEANING	EXAMPLE
(none)	Exactly once	`(title)`
?	Zero or one (optional)	`(note?)`
*	Zero or more	`(tag*)`
+	One or more	`(book+)`
,	Sequence (in order)	`(a, b, c)`
|	Choice (either/or)	`(a | b)`
EMPTY	No content allowed	EMPTY
ANY	Any content	ANY

ATTRIBUTE TYPES

TYPE	MEANING
CDATA	Any character string
ID	Unique identifier in document
IDREF	Reference to an ID value
IDREFS	Space-separated list of IDREFs
NMTOKEN	Valid XML name token
(a|b|c)	Enumeration of allowed values
NOTATION	Refers to a declared notation

COMMON MISTAKES

- Declaring elements out of order in a sequence model — `(a, b, c)` enforces that exact order in the document
- Using `ID` type but not ensuring uniqueness — parsers will error on duplicate ID values
- Expecting DTD to validate data types like numbers or dates — it can't; use XSD for that

TIPS

- Use `!ENTITY` to define repeated strings (company names, copyright text) — a single change updates the whole document
- Prefer external DTDs for shared schemas; internal DTDs are fine for quick one-off validation
- For new projects requiring validation, use XSD instead — it supports namespaces and data types that DTDs cannot express

SUMMARY

- `!ELEMENT` — defines an element and its content model
- `!ATTLIST` — declares attributes, their types, and defaults
- `!ENTITY` — defines reusable text; referenced as `&name;`
- Occurrence: `?` 0–1 · `*` 0–n · `+` 1–n · (none) exactly 1

XML Schema (XSD)

WHAT IS XSD?

XML Schema Definition (XSD) is the W3C standard for describing the structure and data types of an XML document. Unlike DTDs, XSD schemas are themselves valid XML documents, support over 40 built-in data types (string, integer, date, boolean, etc.), and are fully namespace-aware. A schema is linked to an XML document via the `xsi:schemaLocation` attribute. XSD uses the namespace prefix `xs:` or `xsd:` bound to `http://www.w3.org/2001/XMLSchema`.

XSD VS DTD ADVANTAGES

- 40+ built-in data types: `xs:integer`, `xs:date`, `xs:boolean`, etc.
- Full namespace support
- XSD is itself XML — parse and generate with standard XML tools
- Supports default and fixed values, min/maxOccurs
- Extensible: derive new types from existing ones
- Supports `xs:any` for open-content models
- Can import/include other schemas

NOTES

- → `xs:sequence` — children must appear in the declared order; use `xs:all` for any order, `xs:choice` for one-of
- → `minOccurs="0"` — makes element optional (default is 1); `maxOccurs="unbounded"` allows unlimited repetitions
- → `xs:complexType` — element that contains children or attributes; `xs:simpleType` — element with text only
- → `xs:restriction` — derives a new type by constraining an existing one; facets include `xs:minLength`, `xs:pattern`, `xs:enumeration`
- → `use="required"` — attribute must be present; `use="optional"` (default) or `use="prohibited"`
- → `type="xs:ID"` — value must be unique in the document, just like DTD's `ID` type

XSD SYNTAX

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- ── Simple element ──────────────────────────────── -->
  <xs:element name="title"  type="xs:string"/>
  <xs:element name="price"  type="xs:decimal"/>
  <xs:element name="inStock" type="xs:boolean"/>

  <!-- ── Complex type with sequence of children ──────── -->
  <xs:complexType name="BookType">
    <xs:sequence>
      <xs:element name="title"  type="xs:string"/>
      <xs:element name="author" type="xs:string" maxOccurs="unbounded"/>
      <xs:element name="price"  type="xs:decimal"/>
      <xs:element name="note"   type="xs:string" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id"   type="xs:ID"     use="required"/>
    <xs:attribute name="lang" type="xs:string" default="en"/>
  </xs:complexType>

  <!-- ── Restriction: constrain a string ─────────────── -->
  <xs:simpleType name="CurrencyCode">
    <xs:restriction base="xs:string">
      <xs:enumeration value="USD"/>
      <xs:enumeration value="EUR"/>
      <xs:enumeration value="GBP"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- ── Root element using our complex type ─────────── -->
  <xs:element name="catalog">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" type="BookType" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

COMMON BUILT-IN TYPES

TYPE	DESCRIPTION
`xs:string`	Any text string
`xs:integer`	Whole number (no decimal)
`xs:decimal`	Arbitrary-precision decimal
`xs:boolean`	`true` or `false`
`xs:date`	YYYY-MM-DD format
`xs:dateTime`	YYYY-MM-DDThh:mm:ss
`xs:anyURI`	URI or URL value
`xs:ID`	Unique identifier in document
`xs:IDREF`	Reference to an xs:ID value

RESTRICTION FACETS

FACET	CONSTRAINS
`xs:minLength`	Minimum string/list length
`xs:maxLength`	Maximum string/list length
`xs:pattern`	Regex pattern match
`xs:enumeration`	Allowed values list
`xs:minInclusive`	Minimum numeric value (≥)
`xs:maxInclusive`	Maximum numeric value (≤)
`xs:totalDigits`	Max significant digits
`xs:fractionDigits`	Max digits after decimal

COMMON MISTAKES

- Confusing `xs:sequence` (ordered) with `xs:all` (any order) — sequence is far more common and enforces child element ordering
- Forgetting `minOccurs="0"` to make an element optional — default is required (minOccurs=1)
- Linking the schema with the wrong namespace URI in `xsi:schemaLocation` — the namespace must exactly match the schema's `targetNamespace`

TIPS

- Name complex types with a `Type` suffix (`BookType`) to distinguish type definitions from element declarations
- Use `xs:pattern` with a regex to validate formats like postal codes, phone numbers, or version strings
- Validate with `xmllint --schema schema.xsd doc.xml` on the command line, or use an IDE plugin for immediate feedback

SUMMARY

- `xs:element` + `xs:complexType` — define structured elements with children
- `xs:sequence` / `xs:choice` / `xs:all` — control child ordering
- `minOccurs` / `maxOccurs` — control repetition (default 1 each)
- `xs:restriction` — constrain a base type with facets like `xs:pattern`, `xs:enumeration`

XPath

axes · node selection · predicates · functions · absolute · relative paths

XPATH — XML PATH LANGUAGE

XPath is the query language for selecting nodes from an XML document. An XPath expression describes a path through the document tree, similar to a filesystem path. Expressions are evaluated relative to a **context node**. A path starting with / is *absolute* (from the document root); one without is *relative* (from the current node). XPath is used natively inside XSLT, XML Schema, and XQuery, and is exposed in most programming languages through DOM APIs and dedicated libraries.

NODE TYPES

- **Element** — a tagged element (`<book>`)
- **Attribute** — an attribute of an element (`@id`)
- **Text** — text content inside an element
- **Comment** — `<!-- … -->`
- **Processing instruction** — `<?target data?>`
- **Document root** — the virtual top node above the root element
- **Namespace** — namespace node on an element

PATH EXPRESSIONS & PREDICATES

EXPRESSION	SELECTS
`/catalog`	Root <catalog> element
`/catalog/book`	All <book> children of <catalog>
`//book`	All <book> elements anywhere in document
`book/title`	<title> children of <book> (relative)
`//@id`	All id attributes anywhere
`book/@lang`	lang attribute of each <book>
`/catalog/book[1]`	First <book> (1-indexed)
`/catalog/book[last()]`	Last <book>
`//book[@id]`	Books that have an id attribute
`//book[@id='bk101']`	Book where id equals bk101
`//book[price>10]`	Books where <price> content > 10
`/catalog/*`	All element children of <catalog>
`/catalog/node()`	All children (elements, text, comments)
`.`	Current (context) node
`..`	Parent of current node

USEFUL XPATH FUNCTIONS

FUNCTION	RETURNS
`text()`	Text node children of context node
`node()`	Any node type
`count(node-set)`	Number of nodes in set
`last()`	Position of last node in context
`position()`	Position of current node (1-based)
`contains(str, sub)`	True if str contains sub
`starts-with(str, pre)`	True if str starts with pre
`normalize-space(str)`	Trims and collapses whitespace
`string(node)`	String value of node
`not(expr)`	Boolean negation
`name()`	Local name of context node
`local-name()`	Name without namespace prefix

XPATH AXES

AXIS	ABBREVIATED	SELECTS
`child::`	`el`	Direct children (default axis)
`attribute::`	`@attr`	Attributes of context node
`descendant::`	`//el`	All descendants at any depth
`descendant-or-self::`	`//`	Context node + all descendants
`parent::`	`..`	Direct parent node
`ancestor::`	—	All ancestors up to root
`ancestor-or-self::`	—	Context node + all ancestors
`following-sibling::`	—	Siblings after context node
`preceding-sibling::`	—	Siblings before context node
`self::`	`.`	The context node itself

COMMON MISTAKES

- Forgetting XPath positions are 1-based: `[1]` is the first node, not `[0]`
- Using `//` at the start of every expression — it scans the entire tree each time and is very slow on large documents; prefer a more specific path
- XPath on namespaced XML without registering prefixes — the engine needs prefix-URI bindings to match namespaced elements

TIPS

- Test XPath expressions interactively in browser DevTools: `$x("//book[@id='bk101']")` in the console
- Use `normalize-space()` around text comparisons to ignore leading/trailing whitespace differences
- Combine predicates: `//book[@lang='en'][price > 10]` — multiple predicates are ANDed together

SUMMARY

- `/` — absolute path from root; no `/` = relative from context node
- `//` — anywhere in document (descendant-or-self)
- `@attr` — select attribute; `[predicate]` — filter a node-set
- Positions are 1-based; `last()` returns final position in context

XSLT

templates · apply-templates · for-each · if · choose · output methods

WHAT IS XSLT?

XSLT (XSL Transformations) is an XML-based language for transforming XML documents into other formats — HTML, plain text, another XML vocabulary, or any text output. An XSLT stylesheet is itself a well-formed XML document using the `xsl:` namespace. Processing works by matching nodes in the source tree with **templates** and outputting their transformed result. XSLT uses **XPath** expressions to select and test nodes. The XSLT processor applies templates recursively until the entire source tree has been processed.

CORE XSLT ELEMENTS

- `xsl:stylesheet` — root element of every XSLT stylesheet
- `xsl:template` — defines a rule matching a set of nodes
- `xsl:apply-templates` — processes child nodes with matching templates
- `xsl:value-of` — outputs the string value of an XPath expression
- `xsl:for-each` — iterates over a node-set
- `xsl:if` — conditional output
- `xsl:choose` / `xsl:when` / `xsl:otherwise` — switch-style conditionals
- `xsl:output` — declares output method and encoding

NOTES

- `match="/"` — matches the document root node (virtual node above the root element); this template fires first
- `xsl:apply-templates select="..."` — hands off processing to whichever template matches each selected node; without `select`, processes all children
- `xsl:value-of select="title"` — outputs the string value of `<title>`; equivalent to XPath `string(title)`
- `test="price > 10"` — XPath expression in `test`; use `>` because `>` would break XML parsing
- `xsl:choose` — like a switch statement; `xsl:when` clauses are tested top-to-bottom; `xsl:otherwise` is the default
- `xsl:sort` — sorts the node-set before iteration; `order="descending"` also available; `data-type="number"` for numeric sort
- `xsl:output method="html"` — also accepts `"xml"` and `"text"`; controls serialization of the result

OUTPUT METHODS

METHOD=	OUTPUT
`"html"`	HTML; void elements like not closed
`"xml"`	XML; all elements properly closed
`"text"`	Plain text; all markup stripped

XSLT STYLESHEET — XML TO HTML

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <!-- ── Output format ──────────────────────────────── -->
  <xsl:output method="html" encoding="UTF-8" indent="yes"/>

  <!-- ── Root template — matches document root ──────── -->
  <xsl:template match="/">
    <html><body>
      <h1>Book Catalog</h1>
      <xsl:apply-templates select="catalog/book"/>
    </body></html>
  </xsl:template>

  <!-- ── Template for each book ─────────────────────── -->
  <xsl:template match="book">
    <div>
      <h2><xsl:value-of select="title"/></h2>
      <p>Author: <xsl:value-of select="author"/></p>

      <!-- xsl:if — single condition -->
      <xsl:if test="price &gt; 10">
        <p>Price: <xsl:value-of select="price"/></p>
      </xsl:if>

      <!-- xsl:choose — multiple conditions -->
      <xsl:choose>
        <xsl:when test="@lang = 'en'"><span>English</span></xsl:when>
        <xsl:when test="@lang = 'fr'"><span>French</span></xsl:when>
        <xsl:otherwise><span>Other</span></xsl:otherwise>
      </xsl:choose>
    </div>
  </xsl:template>

  <!-- ── for-each with sort ─────────────────────────── -->
  <xsl:template match="catalog">
    <xsl:for-each select="book">
      <xsl:sort select="title" order="ascending"/>
      <p><xsl:value-of select="title"/></p>
    </xsl:for-each>
  </xsl:template>

</xsl:stylesheet>
```

XSLT ELEMENT QUICK REFERENCE

ELEMENT	KEY ATTRIBUTE(S)	PURPOSE
`xsl:stylesheet`	version, xmlns:xsl	Root of every stylesheet
`xsl:output`	method, encoding, indent	Output serialization
`xsl:template`	match, name, mode	Define a transformation rule
`xsl:apply-templates`	select, mode	Delegate to matching templates
`xsl:call-template`	name	Call a named template
`xsl:value-of`	select	Output string value of XPath
`xsl:copy-of`	select	Copy nodes verbatim
`xsl:for-each`	select	Iterate over a node-set
`xsl:sort`	select, order, data-type	Sort within for-each or apply-templates
`xsl:if`	test	Conditional — no else branch
`xsl:choose`	—	Multi-branch conditional
`xsl:variable`	name, select	Declare an immutable variable
`xsl:param`	name, select	Template parameter

COMMON MISTAKES

- Using `>` or `<` raw inside `test="..."` — XPath operators inside XML attributes must be escaped as `>` / `<`
- Forgetting that `xsl:variable` is immutable — you cannot update a variable inside a loop; use recursion or `xsl:for-each` accumulation patterns instead
- Omitting `xsl:apply-templates` in a template — without it, child nodes are silently dropped from the output

TIPS

- Prefer template matching (`xsl:apply-templates`) over `xsl:for-each` — it produces more modular, reusable stylesheets
- Use `xsl:copy-of` instead of `xsl:value-of` when you need to copy a subtree with its markup intact
- Test stylesheets with `xsltproc input.xml transform.xsl` on the command line for fast iteration

SUMMARY

- `xsl:template match="..."` — rule triggered when a node matches the XPath pattern
- `xsl:apply-templates` — recurse into child nodes; drives the transformation engine
- `xsl:value-of` — output text; `xsl:copy-of` — output node tree
- `xsl:if` (no else) · `xsl:choose/when/otherwise` (multi-branch)

Parsing XML

PARSING MODELS

Three major models exist for parsing XML programmatically. **DOM (Document Object Model)** loads the entire document into a tree in memory — easy to navigate and modify, but memory-intensive for large files. **SAX (Simple API for XML)** streams the document and fires events (startElement, endElement, characters) as it reads — low memory, but read-only and forward-only. **StAX (Streaming API for XML)** is a pull-based streaming model where your code drives parsing iteration — a middle ground that is easier to use than SAX while remaining memory-efficient.

MODEL COMPARISON

- **DOM** — full tree in RAM; random access; read/write; slow on large files
- **SAX** — event-driven; push model; very low memory; read-only; forward-only
- **StAX** — pull streaming; low memory; more control than SAX; Java-native
- Browser JS uses DOM API natively via `DOMParser`
- Python: `xml.etree.ElementTree` = lightweight DOM; `xml.sax` = SAX
- Always check for parse errors — malformed XML throws, not silently fails

PARSING IN PYTHON, JS & JAVA

```
# ── Python: xml.etree.ElementTree (DOM-like) ──────────────
import xml.etree.ElementTree as ET

tree = ET.parse('catalog.xml')
root = tree.getroot()                # <catalog> element

for book in root.findall('book'):
    title = book.find('title').text  # text content
    bid   = book.get('id')           # attribute value
    price = float(book.find('price').text)
    print(bid, title, price)

# XPath-like selection (limited subset)
books = root.findall('.//book[@lang="en"]')

# Build and write XML
new_book = ET.SubElement(root, 'book', id='bk103')
ET.SubElement(new_book, 'title').text = 'New Title'
tree.write('out.xml', encoding='UTF-8', xml_declaration=True)
```

NOTES

- → `ET.parse()` — loads file into memory; use `ET.fromstring(xml_str)` to parse a string directly
- → `.findall('book')` — finds direct children named `book`; `'.//book'` finds at any depth
- → `.find('title').text` — the text content of an element; returns `None` if element is missing — guard against this
- → `.get('id')` — attribute value; returns `None` if attribute absent (use `.get('id', 'default')` for a fallback)
- → `ET.SubElement(parent, tag)` — creates and appends a new child element in one call
- → `xml_declaration=True` — writes `<?xml version='1.0' encoding='UTF-8'?>` at the top of the output file

BROWSER / JS DOM API

METHOD / PROPERTY	DESCRIPTION
`new DOMParser()`	Create a parser instance
`.parseFromString(str, mime)`	Parse XML string; mime: `"text/xml"`
`doc.querySelector(css)`	CSS selector on XML document
`doc.querySelectorAll(css)`	NodeList of all matches
`el.getAttribute(name)`	Read an attribute value
`el.textContent`	All text inside element (incl. descendants)
`el.children`	HTMLCollection of element children
`new XMLSerializer()`	Serialize DOM back to XML string
`.serializeToString(node)`	Node → XML string
`doc.querySelector("parsererror")`	Check for parse errors

PYTHON ELEMENTTREE API

METHOD	DESCRIPTION
`ET.parse(file)`	Parse file → ElementTree object
`ET.fromstring(str)`	Parse string → root Element
`tree.getroot()`	Get root Element of the tree
`el.find(path)`	First match (XPath subset)
`el.findall(path)`	All matches (list)
`el.get(attr)`	Attribute value or None
`el.text`	Text content of element
`el.tag`	Element tag name
`el.attrib`	Dict of all attributes
`ET.SubElement(parent, tag)`	Create and append child
`tree.write(file)`	Serialize to file

COMMON MISTAKES

- Not checking for `None` after `.find()` — if the element is missing, accessing `.text` on `None` throws an `AttributeError`
- Using browser DOMParser without checking for `<parsererror>` — a malformed document still returns a document object, not an exception
- Loading huge XML files with DOM parsers — use SAX or StAX for files over ~50 MB to avoid memory exhaustion

TIPS

- Use Python's `lxml` library for full XPath 1.0 support and much faster parsing than the standard library's `xml.etree`
- In the browser, `doc.querySelector()` with CSS selectors is often simpler than XPath for basic navigation
- For namespaced XML in Python ElementTree, element names appear as `{uri}localname` — prefix with the URI in braces in your selectors

SUMMARY

- DOM — full tree in memory; best for small/medium files with random access
- SAX — event push; best for huge files; read-only forward-only
- Python: `ET.parse()` / `ET.fromstring()` · JS: `new DOMParser().parseFromString()`
- Always guard against missing elements (`None`) and parse errors

Practical XML

XML IN THE REAL WORLD

XML underpins many formats you encounter daily without necessarily thinking of them as "XML." **SVG** graphics, **RSS/Atom** feeds, **SOAP** web service messages, Office Open XML (`.docx`, `.xlsx` internals), Maven `pom.xml`, Android layouts and manifests, and Spring configuration files are all XML vocabularies. Understanding the shared XML foundation means you can read, edit, validate, and transform any of them with the same tools and mental model.

COMMON XML TOOLING

- `xmllint` — validate, format, and XPath-query from the command line
- `xsltproc` — apply XSLT stylesheets from the command line
- VS Code + XML extension (Red Hat) — schema-aware editing and validation
- Oxygen XML Editor — full IDE for XML, XSD, XSLT, and XPath
- Python `lxml` — fast C-backed DOM, full XPath 1.0, XSLT 1.0
- Java JAXB — bind XML to Java objects with annotations
- Online: xmlvalidation.com, freeformatter.com/xml-formatter

NOTES

- → RSS 2.0 is plain XML with no namespace — each `<item>` is one post/episode; `<guid>` should be a permanent unique URL
- → SOAP wraps every request in `<soap:Envelope>` + `<soap:Body>` ; the actual operation is a child of Body in its own namespace
- → SOAP responses follow the same envelope structure; faults go in `<soap:Fault>` inside Body
- → Maven POM must declare `xmlns="http://maven.apache.org/POM/4.0.0"` ; Maven validates against its embedded XSD
- → SVG is XML: you can open any `.svg` file in a text editor, apply XPath, or transform it with XSLT
- → Android `AndroidManifest.xml` and layout files use the `android:` namespace — attributes like `android:id` reference the SDK's namespace URI

REAL-WORLD XML SNIPPETS

```xml
<!-- ── RSS 2.0 feed ──────────────────────────────────────── -->
<rss version="2.0">
  <channel>
    <title>My Blog</title>
    <link>https://example.com</link>
    <description>Latest posts</description>
    <item>
      <title>Post Title</title>
      <link>https://example.com/post-1</link>
      <pubDate>Mon, 01 Jan 2025 00:00:00 GMT</pubDate>
      <guid>https://example.com/post-1</guid>
    </item>
  </channel>
</rss>

<!-- ── SOAP 1.1 request envelope ─────────────────────────── -->
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <GetStockPrice xmlns="http://example.com/stock">
      <StockName>AAPL</StockName>
    </GetStockPrice>
  </soap:Body>
</soap:Envelope>

<!-- ── Maven pom.xml (partial) ──────────────────────────── -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-core</artifactId>
      <version>6.1.0</version>
    </dependency>
  </dependencies>
</project>
```

XML VS JSON

FEATURE	XML	JSON
Syntax	Tags + attributes	Keys + values
Schema	XSD, DTD, Relax NG	JSON Schema
Comments	✔ <!-- -->	✖ not supported
Namespaces	✔ built-in	✖ convention only
Attributes	✔ first-class	✖ key-value only
Mixed content	✔ text + elements	✖ not applicable
Query language	XPath / XQuery	JSONPath / jq
Transform	XSLT	jq / custom code
Verbosity	High	Low
Best for	Documents, SOAP, config	REST APIs, data

XMLLINT COMMAND REFERENCE

COMMAND	PURPOSE
`xmllint file.xml`	Check well-formedness
`xmllint --format file.xml`	Pretty-print / indent
`xmllint --schema s.xsd f.xml`	Validate against XSD
`xmllint --dtdvalid d.dtd f.xml`	Validate against DTD
`xmllint --xpath "//book" f.xml`	Run XPath expression
`xsltproc t.xsl input.xml`	Apply XSLT stylesheet
`xmllint --noout --valid f.xml`	Silent validation (exit code only)

COMMON MISTAKES

- Editing `pom.xml` or Android XML files without namespace awareness — removing or changing `xmlns` declarations breaks tool processing
- Treating RSS/Atom feeds as plain text — they're full XML; parse with an XML parser to handle entities and encoding correctly
- Converting XML to JSON naively — attributes, mixed content, and repeated element names all need special mapping decisions before writing conversion code

TIPS

- Run `xmllint --format file.xml` to canonicalize indentation before committing XML files to version control — cleaner diffs
- Use `xmllint --noout --schema schema.xsd doc.xml` in CI pipelines to validate config XML on every build
- When converting XML → JSON, document your mapping rules for attributes (prefix with `@` ?) and repeated elements (always array?) before writing any code

SUMMARY

- SVG, RSS, SOAP, pom.xml, Android layouts — all standard XML vocabularies
- XML favors documents + namespaces + schema; JSON favors lightweight data APIs
- `xmllint` validates, formats, and XPath-queries from the command line
- Know the XML → JSON mapping conventions before writing conversion logic

XML Security

XXE · billion laughs · external entities · disallow-doctype-decl · defusedxml · XPath injection

THE DTD IS THE ATTACK SURFACE

XML parsers support DTD features that most applications never use: **external entities** that pull in files or URLs, and **entity expansion** that can grow a few bytes into gigabytes. Attackers abuse them through any endpoint that parses XML they control — SOAP APIs, SAML logins, file uploads (`.docx`, `.svg`), RSS imports. **XXE** (XML External Entity) reads server files or makes the server call internal URLs; **billion laughs** exhausts memory. The fix is the same everywhere: **turn off DTD processing** unless you truly need it.

DEFENSE CHECKLIST

- Disallow `<!DOCTYPE>` entirely — the simplest and safest setting
- If DTDs are needed: disable external entities, external DTD loading and XInclude
- Keep entity-expansion limits on (Java secure processing, libxml2 defaults, Expat ≥ 2.4.1)
- Never build XPath or XQuery strings from user input — pass variables
- Treat uploaded Office files and SVGs as XML input — they are parsed too

NOTES

→ `SYSTEM "file:///…"` — external entity; `&xxe;` is replaced by the file's contents in the parsed result

→ `SYSTEM "http://…"` instead of a file → SSRF: the server fetches internal URLs such as cloud metadata

→ `defusedxml` — drop-in wrappers for the stdlib parsers that forbid entity declarations and external access

→ `resolve_entities=False` — lxml keeps `&xxe;` unexpanded; `no_network=True` blocks URL fetches

→ `disallow-doctype-decl` — Xerces/JDK parsers reject any document with a DOCTYPE (OWASP's recommended setting)

→ `FEATURE_SECURE_PROCESSING` — turns on the JDK's entity-expansion and size limits

ATTACK PAYLOADS

```
<!-- XXE: read a local file into the document -->
<?xml version="1.0"?>
<!DOCTYPE user [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<user><name>&xxe;</name></user>

<!-- Billion laughs: 10 levels × 10 refs = 10^9 "lol"s ≈ 3 GB -->
<!DOCTYPE lolz [
  <!ENTITY lol  "lol">
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!-- … lol3 … lol8 the same way … -->
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<lolz>&lol9;</lolz>
```

SAFE PARSING — PYTHON

```
import defusedxml.ElementTree as ET        # pip install defusedxml
tree = ET.parse("upload.xml")              # raises EntitiesForbidden on <!ENTITY>

from lxml import etree
parser = etree.XMLParser(resolve_entities=False, no_network=True, load_dtd=False)
tree = etree.parse("upload.xml", parser)
```

SAFE PARSING — JAVA

```
DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
f.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
f.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
f.setXIncludeAware(false);
f.setExpandEntityReferences(false);
Document doc = f.newDocumentBuilder().parse(input);
```

ATTACKS & DEFENSES

ATTACK	EFFECT	DEFENSE
XXE (file)	reads server files	disable DTDs / external entities
XXE (SSRF)	server calls internal URLs	same + `no_network`
Blind / OOB XXE	exfiltrates via parameter entities	disable external DTD loading
Billion laughs	exponential memory use	expansion limits or no DTD
XInclude	includes local files	`setXIncludeAware(false)`
XPath injection	`' or '1'='1` bypasses filters	XPath variables, not string concat

SAFE SETTINGS BY PLATFORM

PLATFORM	SETTING
Python stdlib	use `defusedxml.*` equivalents
Python lxml	`resolve_entities=False, no_network=True`
Java DOM / SAX	`disallow-doctype-decl` = true
.NET	`DtdProcessing.Prohibit`, `XmlResolver = null` (default since 4.5.2)
PHP	libxml ≥ 2.9 safe by default; never pass `LIBXML_NOENT`
Browsers	`DOMParser` never loads external entities

COMMON MISTAKES

- Assuming JSON-only APIs are safe — many frameworks still accept `Content-Type: application/xml` on the same endpoint
- Parsing SAML responses, `.docx` or `.svg` uploads with default parser settings — they are XML documents too
- Passing `LIBXML_NOENT` (PHP) or `--noent` (xmllint) "to expand entities" — that is exactly what enables XXE
- Building XPath with string concatenation: `"//user[name='" + name + "']"` — use bound variables

TIPS

- Search the codebase for every parser factory (`DocumentBuilderFactory`, `XMLInputFactory`, `etree.XMLParser`) and harden each one
- Test endpoints with the XXE payload above pointing at a harmless file you control
- OWASP's **XXE Prevention Cheat Sheet** lists the exact flags for each Java, .NET and C/C++ parser
- If you only need data, consider JSON for new APIs — it has no entity system to attack

SUMMARY

- `XXE` — external entities read files / call URLs
- `billion laughs` — nested entities exhaust memory
- `disallow-doctype-decl` / `DtdProcessing.Prohibit` — reject DTDs
- `defusedxml` — safe drop-in for Python
- `XPath variables` — never concatenate user input

XSLT — Variables, Keys & Grouping

THE NEXT STEP AFTER TEMPLATES

Once basic templates work, most real transforms need four more tools. **Variables** hold computed values (they never change after being set). **Parameters** let callers pass values in from the command line. **Keys** build an index for fast lookups by value — like a join. **Grouping** turns flat records into nested summaries. XSLT 2.0/3.0 has `xsl:for-each-group`; XSLT 1.0 (browsers, xsltproc, lxml, .NET) needs the **Muenchian** key trick to do the same.

INPUT USED BELOW

- `<orders>` with `<order id customer total/>` children
- `acme` has orders 1 and 3 (40 + 25); `globex` has order 2 (15)
- Goal: one `<customer>` per name with order count and sum
- Output: `<customer name="acme" count="2" sum="65"/>`, then globex (1, 15)

NOTES

- `xsl:param` — a variable with a default that callers can override
- `{…}` in a literal attribute — attribute value template; the XPath inside is evaluated
- `group-by` — one group per distinct value; `current-group()` and `current-grouping-key()` inside
- `xsl:key` + `key()` — index orders by customer; `key('by-cust', 'acme')` returns both acme orders
- `generate-id()` — unique ID per node; equal only when it is the same node, so it picks each group's first member

XSLT 3.0 — XSL:FOR-EACH-GROUP

```
<xsl:stylesheet version="3.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:param name="minTotal" select="0"/>          <!-- set from command line -->

  <xsl:template match="/orders">
    <report>
      <xsl:for-each-group select="order[@total &gt;= $minTotal]" group-by="@customer">
        <xsl:sort select="current-grouping-key()"/>
        <customer name="{current-grouping-key()}"
                  count="{count(current-group())}"
                  sum="{sum(current-group()/@total)}"/>
      </xsl:for-each-group>
    </report>
  </xsl:template>
</xsl:stylesheet>
```

XSLT 1.0 — MUENCHIAN GROUPING WITH XSL:KEY

```
<xsl:key name="by-cust" match="order" use="@customer"/>

<xsl:template match="/orders">
  <report>
    <!-- keep only the FIRST order for each customer -->
    <xsl:for-each select="order[generate-id() = generate-id(key('by-cust', @customer)[1])]">
      <xsl:sort select="@customer"/>
      <xsl:variable name="group" select="key('by-cust', @customer)"/>
      <customer name="{@customer}" count="{count($group)}" sum="{sum($group/@total)}"/>
    </xsl:for-each>
  </report>
</xsl:template>
```

QUICK SYNTAX

ELEMENT / FUNCTION	USE
`<xsl:variable name="v" select="…"/>`	computed value, read as $v
`<xsl:param name="p" select="default"/>`	overridable input
`<xsl:key name="k" match="…" use="…"/>`	declare an index (top level)
`key("k", value)`	look up nodes by indexed value
`<xsl:apply-templates mode="m"/>`	process with a separate template set
`xsl:for-each-group group-by / group-adjacent`	grouping (2.0+)

XSLT VERSIONS & PROCESSORS

PROCESSOR	VERSION
Browsers (XSLTProcessor)	1.0
xsltproc / libxslt, lxml	1.0
.NET XslCompiledTransform	1.0
Saxon HE / PE / EE	3.0
SaxonJS (browser / Node)	3.0

COMMON MISTAKES

- Trying to update a variable inside a loop — variables are immutable; compute with XPath (`sum()`, `count()`) or recursion
- Writing `select="USD"` — that selects child elements named `USD`; a string needs `select="'USD'"`
- Using `for-each-group` in a 1.0 processor — it errors or is ignored; check `system-property('xsl:version')`
- Declaring `xsl:key` inside a template — keys must be top-level children of `xsl:stylesheet`

TIPS

- Use keys for any repeated lookup ("find the product for this order line") — much faster than `//product[@id = $x]`
- Prefer `apply-templates` with modes over big `xsl:choose` blocks — each case becomes its own template
- Install Saxon HE (free) when you need 2.0/3.0 features; keep 1.0 only where the processor is fixed
- Pass params from the shell: `xsltproc --stringparam minTotal 20 report.xsl orders.xml` or Saxon `… -xsl:report.xsl minTotal=20`

SUMMARY

- `xsl:variable` / `xsl:param` — immutable values; params overridable
- `xsl:key` + `key()` — indexed lookups
- `mode` — multiple template sets for the same nodes
- `xsl:for-each-group` — grouping in 2.0+
- `Muenchian grouping` — `generate-id()` + `key()` in 1.0

XPath 2.0 / 3.1 & XQuery Basics

sequences · if / then / else · for · let · ! · || · => · maps & arrays · FLWOR

BEYOND XPATH 1.0

XPath 1.0 (Sheet 07) works on node-sets, strings, numbers and booleans. **XPath 2.0** replaced node-sets with **sequences** and added typed values (dates, durations), `if`, `for` and regular expressions. **3.0** added `let`, the `!` map operator and `||` string concatenation; **3.1** added **maps, arrays and JSON**. **XQuery** is the query language built on the same expression syntax, with **FLWOR** expressions (`for`, `let`, `where`, `order by`, `return`) that work like SQL over XML. These need a 2.0+ engine such as **Saxon** or **BaseX** — browsers, lxml and `xmllint` stop at 1.0.

SAMPLE DATA

- `<books>` with `<book year="…">` children holding `title`, `author`, `price`
- "Dune" · Herbert · 1965 · 9.99
- "The Dispossessed" · Le Guin · 1974 · 12.50
- "Neuromancer" · Gibson · 1984 · 8.75

NOTES

- **Sequence** — an ordered list of items (nodes or values); `(1, 2, 3)`; no nesting
- `for … return` — maps over a sequence; `$b/price * 2` runs once per book
- `if (…) then … else …` — `else` is required
- `matches` / `replace` / `tokenize` — regex functions; `$1` back-references in `replace`
- `xs:date` + duration — real date arithmetic; strings must be cast first
- `!` — simple map: evaluate the right side once per item on the left
- `||` — string concatenation; numbers are converted automatically
- `=>` — arrow: passes the left value as the first argument
- `?key` — lookup in a map or array; arrays index from **1**
- FLWOR `group by` — re-binds `$b` to the whole group inside `return` (XQuery 3.0)

XPATH 2.0 – 3.1 EXPRESSIONS

```
(: ── 2.0 ── :)
string-join(//author, ", ")                 (: "Herbert, Le Guin, Gibson" :)
//book[@year < 1970]/title/string()         (: "Dune" :)
for $b in //book return $b/price * 2        (: 19.98, 25, 17.5 :)
if (count(//book) > 2) then "many" else "few"     (: "many" :)
avg(//book/price)                           (: 10.413… :)
matches("978-0441013593", "^\d{3}-\d{10}$")  (: true :)
replace("2026-10-23", "(\d+)-(\d+)-(\d+)", "$3/$2/$1")   (: "23/10/2026" :)
xs:date("2026-10-23") + xs:dayTimeDuration("P30D")   (: 2026-11-22 :)

(: ── 3.0 ── :)
//book ! upper-case(title)                  (: "DUNE", "THE DISPOSSESSED", ... :)
"Dune" || " (" || 1965 || ")"               (: "Dune (1965)" :)
let $n := count(//book) return $n * 10      (: 30 :)

(: ── 3.1 ── :)
//book => count()                           (: 3  — same as count(//book) :)
map { "a": 1, "b": 2 }?b                    (: 2 :)
[10, 20, 30]?2                              (: 20  (arrays are 1-based) :)
parse-json('{"tags": ["x", "y"]}')?tags?1   (: "x" :)
```

XQUERY — FLWOR

```
for $b in doc("books.xml")//book
let $y := xs:integer($b/@year)
where $y >= 1970
order by $b/price descending
return <item year="{$y}">{ $b/title/text() }</item>
(: <item year="1974">The Dispossessed</item>
   <item year="1984">Neuromancer</item> :)

for $b in //book
group by $decade := $b/@year idiv 10 * 10
return <decade start="{$decade}" books="{count($b)}"/>
```

WHAT EACH VERSION ADDED

VERSION	YEAR	HIGHLIGHTS
XPath 1.0	1999	node-sets, axes, ~27 functions
XPath 2.0	2007	sequences, types, `if`, `for`, regex, dates
XPath 3.0	2014	`let`, `!`, `||`, inline functions
XPath 3.1	2017	maps, arrays, =>, ?, JSON
XQuery 1.0 / 3.1	2007 / 2017	FLWOR, constructors, modules

ENGINES

ENGINE	SUPPORTS
Saxon HE (Java, .NET, C)	XPath / XQuery / XSLT 3.1
BaseX	XQuery 3.1 database + GUI
eXist-db	XQuery 3.1 database
elementpath (Python)	XPath 1.0–3.1
lxml, xmllint, browsers	XPath 1.0 only
Java javax.xml.xpath	XPath 1.0 only

1.0 VS 2.0+ HABITS

XPATH 1.0	2.0+
`concat(a, b, c)`	`a || b || c`
`translate(x, "abc…", "ABC…")`	`upper-case(x)`
`contains() / starts-with()`	`matches(x, regex)`
`sum(x) div count(x)`	`avg(x)`
`no loops`	`for $x in … return …`

COMMON MISTAKES

- Running 2.0+ expressions in lxml, `xmllint` or a browser — they report "invalid expression"; use Saxon or elementpath
- Comparing a string attribute to a date without casting — use `xs:date(@due) < current-date()`
- Using `=` to test one value against a sequence and expecting "all" — `=` is true if **any** item matches; use `every $x in … satisfies`
- Indexing arrays from 0 — XPath arrays and sequences start at 1

TIPS

- Try expressions interactively in the BaseX GUI or `java -cp saxon-he.jar net.sf.saxon.Query -qs:"…"`
- Use `parse-json()` / `json-doc()` (3.1) to query JSON with the same language as your XML
- Prefer `!` over `for … return` for simple one-line mappings
- Declare types on variables in XQuery (`let $y as xs:integer := …`) to catch bad data early

SUMMARY

- `sequences` — 2.0+ data model; replace node-sets
- `for` / `let` / `if` — expressions inside XPath
- `!` / `||` / `=>` — 3.x operators: map, concat, arrow
- `map {}` / `[]` / `?` — 3.1 maps, arrays, lookup
- `FLWOR` — XQuery's for-let-where-order-return

XML Formats in the Wild

plist · .docx / .xlsx (OOXML) · sitemap.xml · Atom · .csproj · JUnit XML · GPX / KML

YOU ALREADY USE THESE

Beyond the formats on Sheet 10 (RSS, SOAP, SVG, Maven, Android), a lot of everyday files are XML under the hood: macOS and iOS **property lists**, Office documents (**OOXML** — a ZIP of XML parts), **sitemaps** for search engines, **Atom** feeds, .NET **project files**, **JUnit XML** test reports that every CI system reads, and **GPX/KML** for maps and GPS. Each is a small vocabulary with a fixed root element and (usually) a namespace — once you know that, XPath, XSLT and `xmllint` work on all of them.

SHARED PATTERNS

- Find the root element and its `xmlns` first — XPath queries must use that namespace
- Many "binary" formats are ZIPs of XML: `.docx`, `.xlsx`, `.pptx`, `.epub`, `.odt`
- Pretty-print to explore: `xmllint --format file.xml | less`

NOTES

- plist — values are typed by element: `<string>`, `<integer>`, `<real>`, `<true/>`, `<array>`, `<dict>`, `<date>`
- Most plists on disk are **binary**; `plutil` converts both ways; Python has `plistlib`
- sitemap — up to 50,000 URLs / 50 MB per file; list several files in a `<sitemapindex>`
- Atom vs RSS — Atom has a namespace, required `id` / `updated` / `author`, and RFC 3339 dates
- JUnit XML — `tests`, `failures`, `errors`, `skipped` on `<testsuite>`; GitHub, GitLab and Jenkins all read it
- `local-name()` — match elements regardless of namespace when you don't want to bind a prefix

FIVE SMALL VOCABULARIES

```
<!-- Info.plist (Apple) — <key> followed by its value -->
<plist version="1.0"><dict>
  <key>CFBundleName</key>                <string>MyApp</string>
  <key>CFBundleShortVersionString</key> <string>2.1.0</string>
  <key>UIRequiresFullScreen</key>        <true/>
</dict></plist>

<!-- sitemap.xml -->
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>https://example.com/docs</loc><lastmod>2026-10-01</lastmod></url>
</urlset>

<!-- Atom feed (id, title, updated, author are required) -->
<feed xmlns="http://www.w3.org/2005/Atom">
  <id>urn:uuid:60a76c80-d399-11d9-b93c-0003939e0af6</id>
  <title>Release Notes</title>  <updated>2026-10-23T09:00:00Z</updated>
  <author><name>Dev Team</name></author>
  <entry><id>tag:example.com,2026:v2.1</id><title>v2.1</title>
    <updated>2026-10-23T09:00:00Z</updated>
    <link rel="alternate" href="https://example.com/v2.1"/></entry>
</feed>

<!-- JUnit XML test report -->
<testsuite name="api" tests="2" failures="1" time="0.42">
  <testcase classname="users" name="creates user" time="0.10"/>
  <testcase classname="users" name="rejects bad email" time="0.32">
    <failure message="expected 422, got 200"/>
  </testcase>
</testsuite>
```

PEEKING INSIDE

```
plutil -convert xml1 -o - Info.plist          # binary plist → XML (macOS)
plutil -p Info.plist                          # readable dump
unzip -l report.docx                          # list the XML parts
unzip -p report.docx word/document.xml | xmllint --format -
xmllint --xpath "count(//*[local-name()='url'])" sitemap.xml
```

FORMAT GUIDE

FORMAT	ROOT (NAMESPACE)	WHERE
plist	<plist> (none)	macOS / iOS app settings, Info.plist
.docx	<w:document> (wordprocessingml)	word/document.xml inside the ZIP
.xlsx	<worksheet> (spreadsheetml)	xl/worksheets/sheet1.xml
sitemap	<urlset> (sitemaps.org 0.9)	site root, listed in robots.txt
Atom	<feed> (w3.org/2005/Atom)	blogs, release feeds
.csproj	<Project Sdk=…> (none)	.NET projects, Directory.Build.props
JUnit XML	<testsuites> / <testsuite>	CI test reports
GPX	<gpx> (topografix GPX/1/1)	GPS tracks, fitness apps
KML	<kml> (opengis kml/2.2)	Google Earth, map layers

INSIDE A .DOCX / .XLSX

PART	HOLDS
[Content_Types].xml	media type of every part
word/document.xml	body: w:p paragraph → w:r run → w:t text
word/styles.xml	paragraph and character styles
xl/sharedStrings.xml	every unique string, by index
xl/worksheets/sheet1.xml	<c r="A1" t="s"><v>0</v></c> → string #0

COMMON MISTAKES

- Editing a binary plist in a text editor — convert it with `plutil -convert xml1` first (and back if required)
- Searching `.docx` text with grep — Word splits words across several `w:r` runs; join `w:t` text first
- Writing KML coordinates as `lat,lon` — KML uses **longitude first**: `-93.6,42.0,0`
- Reading `.xlsx` cell values directly — `t="s"` cells hold an index into `sharedStrings.xml`, not the text

TIPS

- Use libraries for Office files (`python-docx`, `openpyxl`, Open XML SDK) — the raw XML is easy to break
- Point your CI's test-report step at JUnit XML from any language — pytest `--junitxml`, Jest `jest-junit`
- Validate sitemaps against the official XSD before submitting them to search engines
- Use `plistlib.load()` in Python — it reads XML and binary plists the same way

SUMMARY

- `plist` — `<key>` + typed value pairs; often binary on disk
- `OOXML` — ZIP of XML parts; text lives in `w:t`
- `sitemap` / `Atom` — namespaced web vocabularies
- `.csproj` / `JUnit XML` — build and CI files
- `GPX` / `KML` — geo data; KML is lon,lat

Sheet 15

Quick Reference — Syntax & Validation

well-formedness · elements & attributes · entities & CDATA · namespaces · DTD · XSD types & structure

DOCUMENT SYNTAX

SYNTAX	DOES	#
`<?xml version="1.0"?>`	XML declaration (first line)	01
`<root>…</root>`	exactly one root element	01
`<a/>`	empty element	02
`<!-- note -->`	comment (no -- inside)	01
`<?target data?>`	processing instruction	01
`encoding="UTF-8"`	default when omitted	01

WELL-FORMED RULES

RULE	DETAIL	#
Close every tag	or self-close ` `	01
Nest properly	`<a><b></b></a>`	01
Case-sensitive	`<Item>` ≠ `<item>`	02
Quote attributes	`id="1"` or `id='1'`	02
Unique attributes	no repeats on one element	02
Names	start with letter or _, no `xml…`	02

TEXT & ENTITIES

SYNTAX	MEANS	#
`< >`	< and >	03
`&`	&	03
`" '`	quotes in attributes	03
`© / ©`	character reference	03
`<![CDATA[ … ]]>`	raw text, no escaping	03
`xml:space="preserve"`	keep whitespace	03

NAMESPACES

SYNTAX	DOES	#
`xmlns="uri"`	default namespace	04
`xmlns:x="uri"`	bind prefix x	04
`<x:item>`	qualified name	04
Attributes	unprefixed = no namespace	04
`xmlns=""`	undeclare default	04
`xml:` prefix	reserved, always bound	04

DTD

DECLARATION	DOES	#
`<!DOCTYPE r SYSTEM "r.dtd">`	link external DTD	05
`<!ELEMENT a (b, c*)>`	content model	05
`(#PCDATA) / EMPTY / ANY`	text / none / anything	05
`? * + |`	optional, 0+, 1+, choice	05
`<!ATTLIST a id ID #REQUIRED>`	attribute rules	05
`<!ENTITY co "Acme">`	custom entity `&co;`	05

XSD STRUCTURE

ELEMENT	DOES	#
`xs:element`	declare an element	06
`xs:complexType`	children + attributes	06
`xs:sequence / xs:choice`	ordered / one of	06
`xs:attribute use="required"`	required attribute	06
`minOccurs / maxOccurs`	`0`, `unbounded`	06
`xsi:schemaLocation`	link instance to schema	06

XSD TYPES

TYPE / FACET	DOES	#
`xs:string / xs:int`	built-in types	06
`xs:decimal / xs:boolean`	more built-ins	06
`xs:date / xs:dateTime`	ISO 8601 values	06
`xs:restriction`	derive a simple type	06
`xs:enumeration`	allowed values	06
`xs:pattern / maxLength`	regex / length facets	06

DTD VS XSD

FEATURE	DTD	XSD
Syntax	own syntax	XML
Data types	text only	40+ built-in
Namespaces	no	yes
Occurrence	? * +	any min / max
Entities	yes	no
Tool support	legacy	modern default

⚠ COMMON MISTAKES

- A bare `&` or `<` in text — escape as `&` / `<` or use CDATA
- Assuming a prefix matters — the namespace URI is what identifies elements
- Expecting a default namespace to apply to attributes

✓ TIPS & BEST PRACTICES

- Check well-formedness first (`xmllint --noout f.xml`), then validate
- Prefer XSD over DTD for typed data and namespaces
- Use attributes for metadata, elements for content

QUICK SUMMARY

- One root, close every tag, quote every attribute
- Five built-in entities: `< > & " '`
- Namespaces = URI + optional prefix
- DTD = structure; XSD = structure + types

Sheet 16

Quick Reference — XPath, XSLT & Parsing

XPath axes & functions · XSLT templates & control · parsers by language · DOM vs SAX vs StAX · xmllint · security

XPATH PATHS

EXPRESSION	SELECTS	#
`/root/item`	absolute path	07
`//item`	any depth	07
`@id`	attribute	07
`item[1] / [last()]`	first / last (1-based)	07
`item[@type="a"]`	predicate filter	07
`.. / .`	parent / current	07

XPATH AXES

AXIS	SELECTS	#
`child::`	children (default)	07
`descendant::`	all below	07
`ancestor::`	all above	07
`following-sibling::`	later siblings	07
`preceding-sibling::`	earlier siblings	07
`attribute::`	same as @	07

XPATH FUNCTIONS

FUNCTION	DOES	#
`text()`	text node	07
`count(//item)`	number of nodes	07
`contains(., "x")`	substring test	07
`starts-with(@id, "a")`	prefix test	07
`normalize-space()`	trim + collapse	07
`sum() / string-length()`	numeric / length	07

XSLT TEMPLATES

ELEMENT	DOES	#
`xsl:stylesheet version="3.0"`	root element	08
`xsl:template match="/"`	rule for a node	08
`xsl:apply-templates`	process children	08
`xsl:value-of select="."`	output text	08
`xsl:copy-of`	deep copy nodes	08
`xsl:output method="html"`	output format	08

XSLT CONTROL

ELEMENT	DOES	#
`xsl:for-each`	loop over nodes	08
`xsl:if test="…"`	condition	08
`xsl:choose / when`	if / else chain	08
`xsl:sort select="@n"`	sort inside loop	08
`xsl:variable`	immutable value	08
`{@id}`	attribute value template	08

PARSERS

LANGUAGE	API	#
Python	`xml.etree.ElementTree`, lxml	09
Java	DOM, SAX, StAX, JAXB	09
JavaScript	`DOMParser` (browser)	09
C#	`XDocument` (LINQ to XML)	09
PHP	SimpleXML, DOMDocument	09
Go	`encoding/xml`	09

DOM VS SAX VS STAX

MODEL	HOW	#
DOM	whole tree in memory	09
SAX	push events, callbacks	09
StAX	pull events, iterator	09
Small files	DOM — easy to query	09
Huge files	SAX / StAX — streaming	09
iterparse	Python streaming	09

TOOLS & SECURITY

ITEM	DOES	#
`xmllint --noout`	well-formed check	10
`xmllint --schema s.xsd`	validate	10
`xmllint --xpath`	query from shell	10
`xsltproc t.xsl f.xml`	run XSLT 1.0	10
XXE	disable external entities	10
`defusedxml`	safe Python parsing	10

⚠ COMMON MISTAKES

- Forgetting namespaces in XPath — `//item` won't match `x:item` without a prefix mapping
- XPath positions start at 1, not 0
- Parsing untrusted XML with external entities enabled (XXE)

✓ TIPS & BEST PRACTICES

- Register namespace prefixes with your XPath engine before querying
- Stream big files with SAX, StAX or `iterparse`
- Keep XSLT templates small and let `apply-templates` drive recursion

QUICK SUMMARY

- XPath: paths, axes, predicates, functions
- XSLT: `template` + `apply-templates` + `value-of`
- DOM for small, streaming for large
- Disable DTDs / external entities on untrusted input